Écrire un analyseur de types en Python

Antoine "entwanne" Rozo





pycon.fr
BIARRITZ 2026

Le typage en Python

Le typage en Python

- Typage fort
  - Lève des exceptions en cas d'incompatibilités
  - Évite les conversions implicites

```
In [2]: 5 + 2

Out[2]: 7

In [3]: '5' + '2'

Out[3]: '52'

In [4]: 5 + '2'

---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[4], line 1
----> 1 5 + '2'

TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

pycon.fr
– BIARRITZ 2026

Le typage en Python

- Mais typage dynamique
  - Le type est déduit au *runtime*

In [5]:
```
value = int(x) if (x := input()) else None
print(value, type(value))
```

```
123
123 <class 'int'>
```

pycon.fr
– BIARRITZ 2026

Le typage en Python

- Mais typage dynamique
  - Le type est déduit au *runtime*

In [5]:
```
value = int(x) if (x := input()) else None
print(value, type(value))
```

```
123
123 <class 'int'>
```

- Il est lié à une valeur, pas une variable

In [6]:
```
value = 5
type(value)
```

Out[6]:
```
int
```

In [7]:
```
value = '5'
type(value)
```

Out[7]:
```
str
```

pycon.fr
– BIARRITZ 2026

Duck-typing 🦆

- Duck-typing : on s'intéresse à l'interface, pas au type
    - « si ça marche comme un canard et fait coin-coin, on peut le considérer comme un canard »

Duck-typing 🦆

- Duck-typing : on s'intéresse à l'interface, pas au type
    - « si ça marche comme un canard et fait coin-coin, on peut le considérer comme un canard »

- Une fonction n'a pas besoin de connaître le type d'un argument, seulement de savoir qu'il sera compatible

```
In [8]: max([1, 2, 3])
Out[8]: 3

In [9]: max(range(10))
Out[9]: 9

In [10]: import random
         max(random.randint(1, 10) for _ in range(10))
Out[10]: 10
```

pycon.fr
– BIARRITZ 2026

Duck-typing 🦆

- Python fournit une fonction `isinstance` pour savoir si une valeur est d'un certain type

```
In [11]: isinstance([1, 2, 3], list)
Out[11]:   True
In [12]: isinstance(range(10), list)
Out[12]:   False
In [13]: isinstance('abcdef', list)
Out[13]:   False
```

Duck-typing 🦆

- Qui fonctionne aussi avec des types abstraits

In [14]:
```
from collections.abc import Iterable
isinstance([1, 2, 3], Iterable)
```

Out[14]:
```
True
```

In [15]:
```
isinstance(range(10), Iterable)
```

Out[15]:
```
True
```

In [16]:
```
isinstance('abcdef', Iterable)
```

Out[16]:
```
True
```

Duck-typing 🦆

- Qui fonctionne aussi avec des types abstraits

```
In [14]: from collections.abc import Iterable
         isinstance([1, 2, 3], Iterable)

Out[14]:   True

In [15]: isinstance(range(10), Iterable)

Out[15]:   True

In [16]: isinstance('abcdef', Iterable)

Out[16]:   True
```

- Cela grâce à des mécanismes de *subclass-hooks* qui ne seront pas abordés ici
    - On retient simplement que le type `Iterable` est abstrait et définit une interface
    - Direction le module `abc` si vous voulez en savoir plus

pycon.fr
BIARRITZ 2026

Typage statique

Typage statique

- Annotations apparues en Python 3
    - Standardisées en annotations de types avec Python 3.5
- Forment une couche facultative de typage statique

pycon.fr
BIARRITZ 2026

Typage statique

- Annotations apparues en Python 3
    - Standardisées en annotations de types avec Python 3.5
- Forment une couche facultative de typage statique

In [17]:
```python
def addition(a: int, b: int) -> int:
    return a + b
```

- La fonction pourrait fonctionner avec des objets `str` mais se limite aux `int`
- Les types abstraits vus précédemment peuvent aussi être utilisés comme annotations

pycon.fr
– BIARRITZ 2026

Analyse dynamique

- Ces annotations sont exposées en Python, notamment à l'aide d'`annotationlib`

In [18]:
```
import annotationlib
annotationlib.get_annotations(addition)
```

Out[18]:
```
{'a': int, 'b': int, 'return': int}
```

pycon.fr
– BIARRITZ 2026

Analyse dynamique

- On peut imaginer les exploiter pour vérifier les types à l'exécution

In [19]:

```python
import inspect

def check_types(func):
    annotations = annotationlib.get_annotations(func)
    sig = inspect.signature(func)

    def wrapper(*args, **kwargs):
        bound_args = sig.bind(*args, **kwargs)
        for name, value in bound_args.arguments.items():
            if not isinstance(value, annotations[name]):
                raise TypeError(f'Invalid type for parameter {name}')

        ret = func(*args, **kwargs)
        if not isinstance(ret, annotations['return']):
            raise TypeError('Invalid return type')

        return ret

    return wrapper
```

pycon.fr
BIARRITZ 2026

Analyse dynamique

- Et l'essayer avec quelques tests

In [20]:
```python
@check_types
def addition(a: int, b: int) -> int:
    return a + b
```

In [21]:
```python
addition(3, 5)
```

Out[21]:
```
8
```

In [22]:
```python
addition('foo', 'bar')
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[22], line 1
----> 1 addition('foo', 'bar')

Cell In[19], line 11, in check_types.<locals>.wrapper(*args, **kwargs)
      7     def wrapper(*args, **kwargs):
      8         bound_args = sig.bind(*args, **kwargs)
      9         for name, value in bound_args.arguments.items():
     10             if not isinstance(value, annotations[name]):
---> 11                 raise TypeError(f'Invalid type for parameter {name}')
     12
     13         ret = func(*args, **kwargs)
     14         if not isinstance(ret, annotations['return']):

TypeError: Invalid type for parameter a
```

pycon.fr
BIARRITZ 2026

Analyse dynamique

- Mais cela se limite aux branches effectivement parcourues lors de l'exécution
- L'usage dynamique des annotations reste utile dans d'autres domaines
    - sérialisation de données avec `pydantic` par exemple

```
class MyModel(pydantic.BaseModel):
    name: str
    value: int
```

pycon.fr
BIARRITZ 2026

Analyse statique

- L'analyse statique permet d'avaluer toutes les branches du programme
    - analyse le code sans l'exécuter
    - ne connaît donc pas toutes les conditions / données

mypy

- Analyseur statique de types

In [23]:
```
%%writefile samples/test_mypy.py
def addition(a: int, b: int) -> int:
    return a + b

x: int = addition(3, 5)
y: float = addition(1, 1)
z: list = addition(0, 0)
addition('foo', 'bar')
```

```
Writing samples/test_mypy.py
```

In [24]:
```
mypy samples/test_mypy.py
```

```
test_mypy.py:6: error: Incompatible types in assignment (expression has type "int", variable has type "list[Any]")
[assignment]
test_mypy.py:7: error: Argument 1 to "addition" has incompatible type "str"; expected "int"  [arg-type]
test_mypy.py:7: error: Argument 2 to "addition" has incompatible type "str"; expected "int"  [arg-type]
Found 3 errors in 1 file (checked 1 source file)
```

pycon.fr
BIARRITZ 2026

Typage structurel 🦆

- Le duck-typing n'est pas vraiment compatible avec un typage statique
    - mais trouve un équivalent avec le typage structurel
    - typage structurel : décrire statiquement une interface sans forcer un quelconque type

In [25]:
```python
import numbers

def addition(a: numbers.Number, b: numbers.Number) -> numbers.Number:
    return a + b
```

pycon.fr
BIARRITZ 2026

Autres fonctionnalités

- Unions de type : `int | float`, `str | None`
- Types composés : `list[int]`, `tuple[str, str]`

Autres fonctionnalités

- Unions de type : `int | float`, `str | None`
- Types composés : `list[int]`, `tuple[str, str]`

- Types génériques

```python
class MyType[T]:
    values: list[T]

obj: MyType[int]
```

pycon.fr
BIARRITZ 2026

Autres fonctionnalités

- Protocoles (typage structurel)

```python
class Iterator[T]:
    def __next__(self) -> T:
        ...

class Iterable[T]:
    def __iter__(self) -> Iterator[T]:
        ...

a: Iterable[int] = [1, 2, 3]
b: Iterable[int] = range(10)
```

pycon.fr
BIARRITZ 2026

Écrire un analyseur de types

pycon.fr
BIARRITZ 2026

Arbre syntaxique (AST)

- Accéder aux annotations sans exécuter le code
  - Module `ast` pour parser un fichier Python

Arbre syntaxique (AST)

- Accéder aux annotations sans exécuter le code
  - Module `ast` pour parser un fichier Python

In [26]:
```
%%writefile samples/test_ast.py
def func(a: int) -> int:
    return a + 1

print(func(5))
```

```
Writing samples/test_ast.py
```

In [27]:
```
import ast
from pathlib import Path

content = Path('samples/test_ast.py').read_text()
tree = ast.parse(content)
print(ast.dump(tree))
```

```
Module(body=[FunctionDef(name='func', args=arguments(args=[arg(arg='a', annotation=Name(id
='int', ctx=Load()))]), body=[Return(value=BinOp(left=Name(id='a', ctx=Load()), op=Add(), r
ight=Constant(value=1)))], returns=Name(id='int', ctx=Load())), Expr(value=Call(func=Name(i
d='print', ctx=Load()), args=[Call(func=Name(id='func', ctx=Load()), args=[Constant(value=
5)])]))])
```

pycon.fr
BIARRITZ 2026

Arbre syntaxique (AST)

- Visiteur (*visitor pattern*) pour parcourir les nœuds

In [28]:
```python
class PrintVisitor(ast.NodeVisitor):
    def visit(self, node):
        print(type(node))
        return super().visit(node)

PrintVisitor().visit(tree)
```

```
<class 'ast.Module'>
<class 'ast.FunctionDef'>
<class 'ast.arguments'>
<class 'ast.arg'>
<class 'ast.Name'>
<class 'ast.Load'>
<class 'ast.Return'>
<class 'ast.BinOp'>
<class 'ast.Name'>
<class 'ast.Load'>
<class 'ast.Add'>
<class 'ast.Constant'>
<class 'ast.Name'>
<class 'ast.Load'>
<class 'ast.Expr'>
<class 'ast.Call'>
<class 'ast.Name'>
<class 'ast.Load'>
<class 'ast.Call'>
<class 'ast.Name'>
<class 'ast.Load'>
<class 'ast.Constant'>
```

pycon.fr
BIARRITZ 2026

Arbre syntaxique (AST)

- Peut être spécialisé par type de nœud

In [29]:
```python
class PrintVisitor(ast.NodeVisitor):
    def visit_FunctionDef(self, node):
        print('Defining', node.name)
        return self.generic_visit(node) # parcours récursif

    def visit_Call(self, node):
        print('Calling', node.func.id)
        return self.generic_visit(node)

PrintVisitor().visit(tree)
```

```
Defining func
Calling print
Calling func
```

- Tous les types de nœuds documentés sur docs.python.org/fr/3/library/ast.html

pycon.fr
BIARRITZ 2026

Analyseur de types

- Construction d'un visiteur pour contrôler les types
  - Commencer par analyser les assignations
  - Nœuds `AnnAssign`, `Name` et `Constant`

Analyseur de types

- Construction d'un visiteur pour contrôler les types
    - Commencer par analyser les assignations
    - Nœuds `AnnAssign`, `Name` et `Constant`
- `Constant` : la valeur Python littérale qui est assignée

In [30]:
```python
class TypingVisitor(ast.NodeVisitor):
    def visit_Constant(self, node) -> object:
        return node.value
```

pycon.fr
BIARRITZ 2026

Analyseur de types

- `Name` : le nom du type utilisé comme annotation
- Première ébauche de gestion d'erreur

In [31]:

```python
import builtins

class TypingVisitor(...):
    [...]

    def add_error(self, title, error, node) -> None:
        print(f'{title} on line {node.lineno}: {error}')

    def visit_Name(self, node) -> type | None:
        if hasattr(builtins, node.id):
            return getattr(builtins, node.id)
        self.add_error('Error', f'{node.id} type not found', node=node)
```

pycon.fr
BIARRITZ 2026

Analyseur de types

- `AnnAssign` : l'assignation annotée en tant que telle

In [32]:

```python
class TypingVisitor(...):
    [...]

    def visit_AnnAssign(self, node):
        if node.value is None:
            return

        value = self.visit(node.value)

        annotation = self.visit(node.annotation)
        if not isinstance(annotation, type):
            self.add_error('Error', f'{annotation!r} is not a type', node=annotation_node)
            return

        if not isinstance(value, annotation):
            self.add_error('Assignment error', f'Trying to assign value of type {type(value).__name__}
```

pycon.fr
BIARRITZ 2026

Analyseur de types

- Exécution sur un fichier de test

In [33]:
```
%%writefile assets/assignment.py
a: int = 0 # OK
b: int = "5" # KO
b: str = "" # OK
c: str = 0 # KO
```

```
Writing assets/assignment.py
```

In [34]:
```
visitor = TypingVisitor()
visitor.visit(ast.parse(Path('assets/assignment.py').read_text()))
```

```
Assignment error on line 2: Trying to assign value of type str to variable b annotated with int
Assignment error on line 4: Trying to assign value of type int to variable c annotated with str
```

Contexte d'analyse

- Gestion du contexte : nom de fichier et numéro de ligne
- Meilleur traitement des erreurs : stockage plutôt qu'affichage

In [35]:

```python
class TypingVisitor(...):
    [...]

    def __init__(self):
        self.filename = None
        self.errors = []

    def visit_file(self, node, filename):
        self.filename = filename
        return self.visit(node)

    def add_error(self, title, error, node=None) -> None:
        parts = [title]
        if self.filename:
            parts.append(f' on {self.filename}')
            if node:
                parts.append(f'::{node.lineno}')
        parts.append(f': {error}')
        self.errors.append(''.join(parts))
```

pycon.fr
BIARRITZ 2026

Point d'entrée

- Construction d'une classe `Typy` pour aider au traitement des fichiers
- Point d'entrée de notre analyseur statique de types

In [36]:

```python
class Typy:
    def __init__(self):
        self.visitor = TypingVisitor()

    def run_tree(self, tree: ast.AST, filename: str) -> None:
        self.visitor.visit_file(tree, filename)

    def run_source(self, source: str, filename: str) -> None:
        self.run_tree(ast.parse(source, filename=filename), filename)

    def run_file(self, file, filename: str | None = None) -> None:
        self.run_source(file.read(), filename or file.name)

    def run_path(self, *paths: str | Path) -> None:
        for path in paths:
            with open(path) as file:
                self.run_file(file)
```

pycon.fr
BIARRITZ 2026

Point d'entrée

- Chargée de collecter et afficher les erreurs

In [37]:
```python
class Typy:
    [...]

    @property
    def errors(self) -> list[str]:
        return self.visitor.errors

    def print_errors(self) -> None:
        for error in self.errors:
            print(error)
```

pycon.fr
BIARRITZ 2026

Premier analyseur

- Fonction utilitaire pour lancer `Typy`

In [38]:
```python
def check_files(*paths):
    typy = Typy()
    typy.run_path(*paths)
    typy.print_errors()
```

In [ ]:
```python
# %load assets/assignment.py
a: int = 0 # OK
b: int = "5" # KO
b: str = "" # OK
c: str = 0 # KO
```

In [40]:
```python
check_files('assets/assignment.py')
```

```
Assignment error on assets/assignment.py::2: Trying to assign value of type str to variable b annotated with int
Assignment error on assets/assignment.py::4: Trying to assign value of type int to variable c annotated with str
```

Modèle objet de Python

pycon.fr
– BIARRITZ 2026

Modèle objet de Python

- Répliquer le modèle objet de Python pour mieux l'analyser



Modèle objet de Python

- Répliquer le modèle objet de Python pour mieux l'analyser
- Une valeur est l'instance d'un type



Modèle objet de Python

- Une valeur est l'instance d'un type

In [41]:

```python
import dataclasses

@dataclasses.dataclass(frozen=True)
class Value:
    type: type

class TypingVisitor(...):
    [...]

    def visit_Constant(self, node) -> Value:
        return Value(type=type(node.value))

    def visit_AnnAssign(self, node):
        if node.value is None:
            return

        value = self.visit(node.value)
        if not isinstance(value, Value):
            return

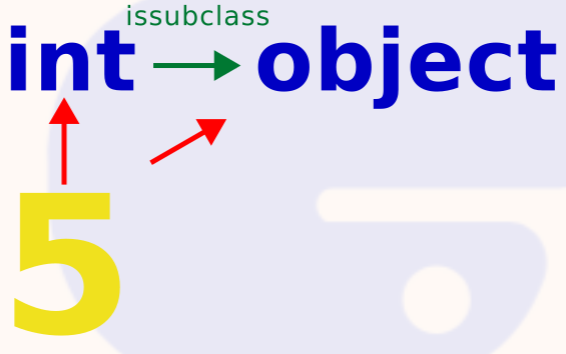
        annotation = self.visit(node.annotation)
        if not isinstance(annotation, type):
            self.add_error('Error', f'{annotation!r} is not a type', node=annotation_node)
            return

        if value.type is not annotation:
            self.add_error('Assignment error', f'Trying to assign value of type {value.type.__name__} t
```

pycon.it
BIARRITZ 2026

Modèle objet de Python

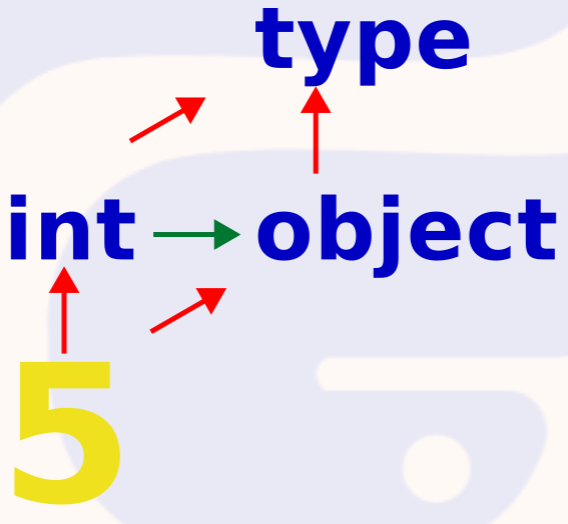
- Chaque type hérite d'`object`



pycon.fr
BIARRITZ 2026

Modèle objet de Python

- Et chaque type est une instance de `type`



pycon.fr
BIARRITZ 2026

Modèle objet de Python

- Et chaque type est une instance de `type`

In [42]:
```python
import typing

class PyObject:
    type: Type

@dataclasses.dataclass(frozen=True)
class Type(PyObject):
    name: str
    type: typing.ClassVar[Type]

    def match(self, value: PyObject) -> bool:
        return value.type is self

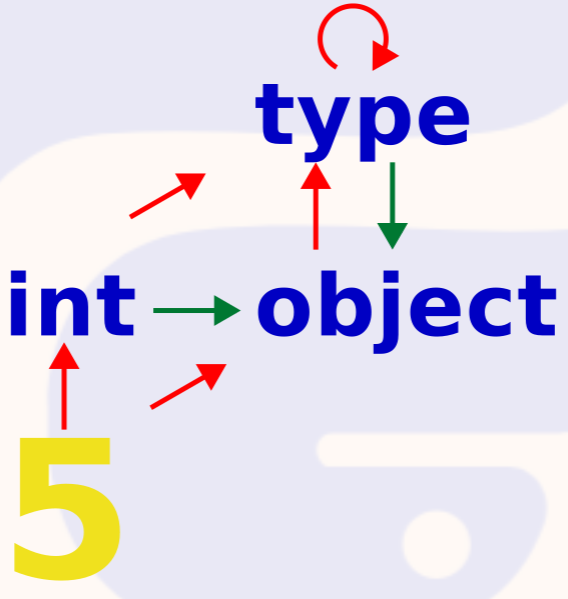
    def __str__(self) -> str:
        return f'<Type:{self.name}>'

@dataclasses.dataclass(frozen=True)
class Value(PyObject):
    type: Type
```

pycon.fr
BIARRITZ 2026

Modèle cyclique

- `type` est lui-même une instance de `type`



pycon.fr
BIARRITZ 2026

Modèle cyclique

- `type` est lui-même une instance de `type`

In [43]:
```
[...]
base_type = Type(name='type')
Type.type = base_type
```

pycon.fr
BIARRITZ 2026

Adaptations au modèle

- Recréons les *builtins* pour correspondre à notre nouveau modèle

In [44]:
```
[...]
builtins = {
    'int': Type(name='int'),
    'float': Type(name='float'),
    'str': Type(name='str'),
}
```

pycon.fr
– BIARRITZ 2026

Adaptations au modèle

- Recréons les *builtins* pour correspondre à notre nouveau modèle

In [44]:
```
[...]
builtins = {
    'int': Type(name='int'),
    'float': Type(name='float'),
    'str': Type(name='str'),
}
```

- Et de quoi représenter l'absence de valeur

In [45]:
```
[...]
NoType = Type('NoType')
NoValue = Value(type=NoType)
```

pycon.fr
BIARRITZ 2026

Adaptations au modèle

- Prise en compte du modèle et des *builtins* dans l'analyseur
- Ainsi que des instructions qui n'ont pas de valeur

In [46]:

```python
class TypingVisitor(...):
    [...]

    def visit(self, node):
        assert node is not None
        value = super().visit(node)
        if not isinstance(value, PyObject):
            raise ValueError(f'Visit call returned a value of type {type(value).__qualname__} for node
        return value

    def generic_visit(self, node):
        super().generic_visit(node)
        return NoValue

    def visit_Constant(self, node):
        return Value(type=builtins[type(node.value).__name__])
```

pycon.fr
BIARRITZ 2026

Adaptations au modèle

- Idem dans la gestion des annotations

In [47]:

```python
class TypingVisitor(...):
    [...]

    def visit_Name(self, node: ast.Name) -> PyObject:
        if node.id in builtins:
            return builtins[node.id]
        self.add_error('Error', f'{node.id} type not found', node=node)
        return NoType

    def visit_AnnAssign(self, node: ast.AnnAssign) -> NoValue:
        if node.value is None:
            return

        value = self.visit(node.value)
        if value is NoValue:
            return NoValue

        annotation = self.visit(node.annotation)
        if not isinstance(annotation, Type):
            self.add_error('Error', f'{annotation!r} is not a type', node=node.annotation)
            return NoValue

        if not annotation.match(value):
            self.add_error('Assignment error', f'Trying to assign value of type {value.type.name} to va

        return NoValue
```

pycon.fr
BIARRITZ 2026

Exécution de l'analyseur

- Pas de changement à l'exécution
  - Mais un modèle plus solide pour la suite

In [ ]:
```python
# %load assets/assignment.py
a: int = 0 # OK
b: int = "5" # KO
b: str = "" # OK
c: str = 0 # KO
```

In [49]:
```python
check_files('assets/assignment.py')
```

```
Assignment error on assets/assignment.py::2: Trying to assign value of type str to variable b annotated with int
Assignment error on assets/assignment.py::4: Trying to assign value of type int to variable c annotated with str
```

pycon.fr
BIARRITZ 2026

Expressions de types

Nettoyage du code

- Nettoyage du visiteur : sortir ce qui peut l'être
    - comme la vérification de type

In [50]:
```python
def check_type(visitor, node):
    type = visitor.visit(node)
    if isinstance(type, Type):
        return type
    visitor.add_error('Error', f'{type!r} is not a type', node=node)
    return NoValue
```

Assignations multiples

- Ou la vérification des annotations
- On en profite pour implémenter la réassignation

In [51]:

```
def check_assign(visitor, target_node, value_node, annotation_node=None):
    assert isinstance(target_node, ast.Name)
    varname = target_node.id
    annotations = visitor.annotations

    value = visitor.visit(value_node) if value_node is not None else NoValue

    if annotation_node is None:
        if varname in annotations:
            annotation = annotations[varname]
        else:
            return
    elif varname in annotations:
        visitor.add_error('Error', f'{varname} variable already defined', node=target_node)
        return
    else:
        try:
            annotation = check_type(visitor, annotation_node)
            if annotation is NoValue:
                return
        except ValueError as e:
            visitor.add_error('Error', str(e), node=target_node)
            return
        annotations[varname] = annotation

    if value is NoValue:
        return

    if not annotation.match(value):
        visitor.add_error('Assignment error', f'Trying to assign value of type {value.type.name} to va
```

Assignations multiples

- Implémentation au niveau du visiteur
  - via la méthode `visit_Assign` complémentaire à `visit_AnnAssign`

In [52]:
```
class TypingVisitor(...):
    [...]

    def __init__(self):
        self.filename = None
        self.annotations = {}
        self.errors = []

    def visit_file(self, node, filename):
        self.filename = filename
        self.annotations.clear()
        return self.visit(node)

    def visit_Assign(self, node: ast.Assign) -> NoValue:
        for target in node.targets:
            check_assign(self, target, node.value)
        return NoValue

    def visit_AnnAssign(self, node: ast.AnnAssign) -> NoValue:
        check_assign(self, node.target, node.value, node.annotation)
        return NoValue
```

pycon.fr – BIARRITZ 2026

Assignations multiples

- Elles sont bien prises en compte par l'analyseur

In [53]:
```
%%writefile assets/assignment.py
a: int = 0 # OK
a = 5 # OK
a = "5" # KO

b: str
b = "" # OK
b = 0 # KO

b: str = "" # KO

c: int = 12 # OK

d: str = 12 # KO
```

```
Overwriting assets/assignment.py
```

In [54]:
```
check_files('assets/assignment.py')
```

```
Assignment error on assets/assignment.py::3: Trying to assign value of type str to variable a annotated with int
Assignment error on assets/assignment.py::7: Trying to assign value of type int to variable b annotated with str
Error on assets/assignment.py::9: b variable already defined
Assignment error on assets/assignment.py::13: Trying to assign value of type int to variable d annotated with str
```

pycon.fr
BIARRITZ 2026

Expressions de types

- Une variable peut-être annotée avec bien plus qu'un simple type
  - Repenser le type comme un ensemble de valeurs autorisées

In [55]:
```
class PyObject:
    type: TypeForm

class TypeForm(PyObject):
    name: str
    type: typing.ClassVar[Type]

    def match(self, value: PyObject) -> bool:
        return value.type <= self

    def __le__(self, type: TypeForm) -> bool:
        return NotImplemented

    def __str__(self) -> str:
        return f'<Type:{self.name}>'
```

In [56]:
```
[...]
TypeForm.type = base_type
```

pycon.fr
BIARRITZ 2026

Expressions de types

- Et le type « concret » comme un cas particulier

In [57]:
```python
@dataclasses.dataclass(frozen=True)
class Value(PyObject):
    type: TypeForm

@dataclasses.dataclass(frozen=True)
class Type(TypeForm):
    name: str

    def __le__(self, rhs: TypeForm) -> bool:
        if isinstance(rhs, __class__):
            return self.name is rhs.name
        return NotImplemented

    def __ge__(self, rhs: TypeForm) -> bool:
        if isinstance(rhs, __class__):
            return self.name is rhs.name
        return NotImplemented
```

pycon.fr
BIARRITZ 2026

Expressions de types

- Ce qui nous amène à redéfinir l'absence de valeur / type

In [58]:
```
import functools

class _NoType(TypeForm):
    @functools.cache
    def __new__(cls):
        return super().__new__(cls)

    @property
    def name(self):
        return '<NoType>'

    def __le__(self, rhs):
        return False

    def __ge__(self, lhs):
        return False

class _NoValue(PyObject):
    @functools.cache
    def __new__(cls):
        return super().__new__(cls)

    @property
    def type(self):
        return NoType

    def __repr__(self):
        return '<NoValue>'
```

In [59]:
```
[...]
NoType = _NoType()
NoValue = _NoValue()
```

Expressions de types

- Et prendre en compte ces expressions de type dans notre vérification

In [60]:
```python
def check_type(visitor, node):
    type = visitor.visit(node)
    if isinstance(type, TypeForm):
        return type
    visitor.add_error('Error', f'{type!r} is not a type', node=node)
    return NoValue
```

pycon.fr
BIARRITZ 2026

Expressions de types

- Toujours pas de changement visible sur l'analyse

In [ ]:
```python
# %load assets/assignment.py
a: int = 0 # OK
a = 5 # OK
a = "5" # KO

b: str
b = "" # OK
b = 0 # KO

b: str = "" # KO

c: int = 12 # OK

d: str = 12 # KO
```

In [62]:
```python
check_files('assets/assignment.py')
```

```
Assignment error on assets/assignment.py::3: Trying to assign value of type str to variable a annotated with int
Assignment error on assets/assignment.py::7: Trying to assign value of type int to variable b annotated with str
Error on assets/assignment.py::9: b variable already defined
Assignment error on assets/assignment.py::13: Trying to assign value of type int to variable d annotated with str
```

pycon.fr
BIARRITZ 2026

Union de types

- Mais permet l'introduction de l'union de types

In [63]:

```python
@dataclasses.dataclass(frozen=True)
class UnionType(TypeForm):
    types: tuple[TypeForm, ...]

    @classmethod
    def from_types(cls, *types):
        return cls(types)

    @property
    def name(self):
        return f'({" | ".join(t.name for t in self.types)})'

    def __le__(self, type):
        return any(subtype <= type for subtype in self.types)

    def __ge__(self, type):
        return any(subtype >= type for subtype in self.types)
```

pycon.fr
BIARRITZ 2026

Union de types

- À intégrer directement via l'opérateur d'union (`|`)

In [64]:

```python
class TypeForm(...):
    [...]

    def __or__(self, rhs):
        return UnionType.from_types(self, rhs)

@dataclasses.dataclass(frozen=True)
class UnionType(...):
    [...]

    def __or__(self, rhs):
        if isinstance(rhs, __class__):
            return __class__.from_types(*self.types, *rhs.types)
        else:
            return __class__.from_types(*self.types, rhs)

    def __ror__(self, lhs):
        # lhs n'est pas un UnionType, sinon on serait passé par __or__
        return __class__.from_types(lhs, *self.types)
```

pycon.fr
BIARRITZ 2026

Union de types

- Qui nécessite de traiter l'opération dans le visiteur

In [65]:
```python
class TypingVisitor(...):
    [...]

    def visit_BinOp(self, node) -> TypeForm:
        left = self.visit(node.left)
        right = self.visit(node.right)

        if isinstance(node.op, ast.BitOr) and isinstance(left, TypeForm) and isinstance(right, TypeForm
            return left | right

        return NoType
```

pycon.fr
BIARRITZ 2026

Union de types

- Et offre plus de possibilités lors du typage

In [66]:
```
%%writefile assets/assignment.py
a: int = 0 # OK
a = 5 # OK
a = "5" # KO

b: str
b = "" # OK
b = 0 # KO

b: str = "" # KO

c: int = 12 # OK

d: str = 12 # KO

e: int | float = 0 # OK
e = 0.5 # OK
f: int | float = 0.1 # OK
g: int | float = '' # KO
```

```
Overwriting assets/assignment.py
```

In [67]:
```
check_files('assets/assignment.py')
```

```
Assignment error on assets/assignment.py::3: Trying to assign value of type str to variable a annotated with int
Assignment error on assets/assignment.py::7: Trying to assign value of type int to variable b annotated with str
Error on assets/assignment.py::9: b variable already defined
Assignment error on assets/assignment.py::13: Trying to assign value of type int to variable d annotated with str
Assignment error on assets/assignment.py::18: Trying to assign value of type str to variable g annotated with (int
| float)
```

pycon.fr
BIARRITZ 2026

Fonctions et opérations

pycon.fr
BIARRITZ 2026

Espaces de noms

- Les définitions se rapportent à un espace de noms particulier

In [68]:

```python
class TypingVisitor(...):
    [...]

    def __init__(self):
        self.filename = None
        self.annotations = {}
        self.namespace = {}
        self.errors = []

    def visit_file(self, node, filename):
        self.filename = filename
        self.annotations.clear()
        self.namespace.clear()
        return self.visit(node)

    def visit_Name(self, node: ast.Name) -> PyObject:
        if node.id in self.namespace:
            return self.namespace[node.id]
        if node.id in builtins:
            return builtins[node.id]
        self.add_error('Error', f'{node.id} variable not found')
        return NoValue
```

pycon.fr
BIARRITZ 2026

Espaces de noms

- L'assignation enregistre donc la variable dans l'espace courant

In [69]:

```
def check_assign(visitor, target_node, value_node, annotation_node=None):
    assert isinstance(target_node, ast.Name)
    varname = target_node.id
    annotations = visitor.annotations

    value = visitor.visit(value_node) if value_node is not None else NoValue

    if annotation_node is None:
        if varname in annotations:
            annotation = annotations[varname]
        else:
            return
    elif varname in annotations:
        visitor.add_error('Error', f'{varname} variable already defined', node=target_node)
        return
    else:
        try:
            annotation = check_type(visitor, annotation_node)
            if annotation is NoValue:
                return
        except ValueError as e:
            visitor.add_error('Error', str(e), node=target_node)
            return
        annotations[varname] = annotation

    if value is NoValue:
        return
    visitor.namespace[varname] = value

    if not annotation.match(value):
        visitor.add_error('Assignment error', f'Trying to assign value of type {value.type.name} to va
```

pycon
BIARRITZ 2026

Espaces de noms

- Pas de changement sur ce fichier, vérification de routine

In [ ]:
```python
# %load assets/assignment.py
a: int = 0 # OK
a = 5 # OK
a = "5" # KO

b: str
b = "" # OK
b = 0 # KO

b: str = "" # KO

c: int = 12 # OK

d: str = 12 # KO

e: int | float = 0 # OK
e = 0.5 # OK
f: int | float = 0.1 # OK
g: int | float = '' # KO
```

In [71]:
```python
check_files('assets/assignment.py')
```

```
Assignment error on assets/assignment.py::3: Trying to assign value of type str to variable a annotated with int
Assignment error on assets/assignment.py::7: Trying to assign value of type int to variable b annotated with str
Error on assets/assignment.py::9: b variable already defined
Assignment error on assets/assignment.py::13: Trying to assign value of type int to variable d annotated with str
Assignment error on assets/assignment.py::18: Trying to assign value of type str to variable g annotated with (int
| float)
```

pycon.fr
BIARRITZ 2026

Définition de fonction

- Une fonction se définit par son nom, ses paramètres et son type de retour
  - Elle doit être capable de reconnaître les types des arguments

In [72]:
```
class Callable(PyObject):
    name: str
    args: tuple[TypeForm, ...]
    returns: TypeForm
    type: typing.ClassVar[Type]

    def match_args(self, *values):
        return len(self.args) == len(values) and all(arg_type.match(value) for arg_type, value in zip(s

    def apply(self, *args: PyObject) -> PyObject:
        return Value(type=self.returns)
```

In [73]:
```
@dataclasses.dataclass(frozen=True)
class Function(Callable):
    name: str
    args: tuple[TypeForm, ...]
    returns: TypeForm
```

pycon.fr
BIARRITZ 2026

Définition de fonction

- Comme tout objet Python, une fonction a un type

In [74]:
```
[...]
function_type = Type(name='function')
Callable.type = function_type
```

pycon.fr
BIARRITZ 2026

Définition de fonction

- On peut donc enregistrer les données d'une fonction quand on rencontre sa définition

In [75]:
```python
class TypingVisitor(...):
    [...]

    def visit_FunctionDef(self, node: ast.FunctionDef) -> NoValue:
        self.namespace[node.name] = Function(
            name=node.name,
            args=tuple(check_type(self, arg.annotation) if arg.annotation else NoType for arg in node.a
            returns=check_type(self, node.returns) if node.returns else NoType,
        )
        return NoValue
```

pycon.fr
BIARRITZ 2026

Appel de fonction

- L'appel doit faire correspondre les types des arguments avec les annotations des paramètres

In [76]:
```
def check_call(visitor, func, args, node):
    if any(arg is NoValue for arg in args):
        return NoValue

    if isinstance(func, Callable) and func.match_args(*args):
        return func.apply(*args)

    visitor.add_error('Call Error', f'cannot call {func.name} with arg types {", ".join(arg.type.name f
    return NoValue
```

pycon.fr
BIARRITZ 2026

Appel de fonction

- Vérification à lancer lorsqu'on rencontre un appel

In [77]:
```python
class TypingVisitor(...):
    [...]

    def visit_Call(self, node: ast.Call) -> PyObject:
        func = self.visit(node.func)
        args = tuple(self.visit(arg) for arg in node.args)
        return check_call(self, func, args, node)
```

pycon.fr
BIARRITZ 2026

Appel de fonction

- Nos fonctions sont maintenant comprises par l'analyseur

In [78]:
```
%%writefile assets/functions.py
def func() -> int:
    return 0

a: int = func() # OK
b: str = func() # KO

def addition(a: int, b: int) -> int:
    return a + b

addition(3, 5) # OK
addition(3, '5') # KO
addition('3', '5') # KO
```

```
Writing assets/functions.py
```

In [79]:
```
check_files('assets/functions.py')
```

```
Assignment error on assets/functions.py::5: Trying to assign value of type int to variable b annotated with str
Call Error on assets/functions.py::11: cannot call addition with arg types int, str
Call Error on assets/functions.py::12: cannot call addition with arg types str, str
```

Opérateurs binaires

- On peut ajouter au cas par cas la gestion des opérateurs
  - Opérateur binaire = 2 opérandes (gauche et droit)
  - Un peu fastidieux pour le moment

In [80]:

```python
def check_operation(visitor, operator, left, right, node):
    if left is NoValue or right is NoValue:
        return NoValue

    if operator is ast.BitOr and isinstance(left, TypeForm) and isinstance(right, TypeForm):
        return left | right

    if operator is ast.Add and left.type is builtins['int'] and right.type is builtins['int']:
        return Value(type=builtins['int'])

    if operator is ast.Add and left.type is builtins['str'] and right.type is builtins['str']:
        return Value(type=builtins['str'])

    if operator is ast.Add and left.type is builtins['float'] and right.type in (builtins['float'], bu
        return Value(type=builtins['float'])

    visitor.add_error('Expression error', f'Incompatible operation {operator.__name__} for values of ty
    return NoValue
```

pycon.fr
BIARRITZ 2026

Opérateurs binaires

- Et les brancher à notre visiteur

In [81]:
```python
class TypingVisitor(...):
    [...]

    def visit_BinOp(self, node):
        left = self.visit(node.left)
        right = self.visit(node.right)
        return check_operation(self, type(node.op), left, right, node)
```

pycon.fr
~ BIARRITZ 2026

Opérateurs binaires

- Ce qui permet de bien reconnaître les opérations

In [82]:
```
%%writefile assets/operations.py
a: int = 3 + 5 # OK
a = 1 + 2 + 3 + 4 # OK
a = 3 + '5' # KO

b: str = 'a' + 'b' # OK
b = 3 + 2 # KO
```

```
Writing assets/operations.py
```

In [83]:
```
check_files('assets/operations.py')
```

```
Expression error on assets/operations.py::3: Incompatible operation Add for values of types int and str
Assignment error on assets/operations.py::6: Trying to assign value of type int to variable b annotated with str
```

pycon.fr
BIARRITZ 2026

Types et classes

pycon.fr
BIARRITZ 2026

Appeler les types

- Un type peut être vu comme une *fonction* renvoyant une valeur du type demandé

In [84]:

```python
def check_call(visitor, func, args, node):
    if any(arg is NoValue for arg in args):
        return NoValue

    if isinstance(func, Callable) and func.match_args(*args):
        return func.apply(*args)

    if isinstance(func, TypeForm):
        return Value(type=func)

    visitor.add_error('Call Error', f'cannot call {getattr(func, 'name', '')} with arg types {", ".joi
    return NoValue
```

pycon.fr
BIARRITZ 2026

Appeler les types

- Le premier prototype fonctionne comme attendu

In [85]:
```
%%writefile assets/type_functions.py
a: int = 3 + int('5') # OK
b: str = str(3) + '5' # OK

b = str(3) + int('5') # KO
```

```
Writing assets/type_functions.py
```

In [86]:
```
check_files('assets/type_functions.py')
```

```
Expression error on assets/type_functions.py::4: Incompatible operation Add for values of types str and int
```

Définition de classe

- On étend alors le concept aux types personnalisés, avec la définition de classes
    - Une classe se définit par son nom et ses parents

In [87]:

```python
@dataclasses.dataclass(frozen=True)
class Type(...):
    [...]
    name: str
    bases: tuple[TypeForm, ...] = ()

    def __le__(self, rhs):
        if self.bases:
            return any(base <= rhs for base in self.bases)
        if isinstance(rhs, __class__):
            return self.name is rhs.name
        return NotImplemented

    def __ge__(self, rhs):
        if self.bases:
            return any(base >= rhs for base in self.bases)
        if isinstance(rhs, __class__):
            return self.name is rhs.name
        return NotImplemented
```

pycon.fr
~ BIARRITZ 2026

Définition de classe

- La classe est alors enregistrée lorsqu'on rencontre sa définition

In [88]:
```python
class TypingVisitor(...):
    [...]

    def visit_ClassDef(self, node):
        class_def = Type(
            name=node.name,
            bases=tuple(check_type(self, base) for base in node.bases),
        )

        self.namespace[node.name] = class_def
        return NoValue
```

pycon.fr
BIARRITZ 2026

Définition de classe

- Cela nous autorise à utiliser nos propres types dans les annotations

In [89]:
```
%%writefile assets/classes.py
class A:
    pass

a: A = A() # OK
b: int = A() # KO

c: A = 0 # KO

class D(int):
    pass

d: int = D() # OK

e: A | int = A() # OK
f: A | int = 5 # OK
g: A | int = '' # KO
h: A | int = D() # OK

i: D | int = A() # KO
j: D | int = D() # OK
k: D | int = 5 # OK
l: D | int = '' # KO
```

```
Writing assets/classes.py
```

In [90]:
```
check_files('assets/classes.py')
```

```
Assignment error on assets/classes.py::5: Trying to assign value of type A to variable b annotated with int
Assignment error on assets/classes.py::7: Trying to assign value of type int to variable c annotated with A
Assignment error on assets/classes.py::16: Trying to assign value of type str to variable g annotated with (A | in
t)
Assignment error on assets/classes.py::19: Trying to assign value of type A to variable i annotated with (D | int)
Assignment error on assets/classes.py::22: Trying to assign value of type str to variable l annotated with (D | in
t)
```

pycon fr
BIARRITZ 2026

Méthodes

- Une classe peut aussi contenir des méthodes
  - Les méthodes sont simplement des fonctions liées à une instance
  - Elles reçoivent cette instance en premier argument

In [91]:

```
@dataclasses.dataclass(frozen=True)
class Method(Callable):
    function: Callable
    value: PyObject

    @property
    def name(self):
        return self.function.name

    @property
    def args(self):
        self_arg, *args = self.function.args
        assert self_arg is self.value.type
        return tuple(args)

    @property
    def returns(self):
        return self.function.returns

    def apply(self, *values):
        return self.function.apply(self.value, *values)
```

pycon.fr
BIARRITZ 2026

Méthodes

- On permet à tout *callable* d'être associé à une instance pour former une méthode

In [92]:
```python
class Callable(...):
    [...]

    def bind(self, value):
        return Method(function=self, value=value)
```

pycon.fr
BIARRITZ 2026

Définition de méthodes

- Les méthodes se rapportent à la classe (type) dans laquelle elles sont défininies

In [93]:
```python
@dataclasses.dataclass(frozen=True)
class Type(...):
    [...]

    name: str
    bases: tuple = ()
    methods: dict = dataclasses.field(repr=False, default_factory=dict)
```

pycon.fr
BIARRITZ 2026

Définition de méthodes

- Et sont donc découvertes lors de la définition de la classe, dans un espace de noms dédié

In [94]:

```python
from collections import ChainMap

class TypingVisitor(...):
    [...]

    def visit_ClassDef(self, node):
        class_def = Type(
            name=node.name,
            bases=tuple(check_type(self, base) for base in node.bases),
        )

        self.namespace[node.name] = class_def
        current_ns, current_ann, class_ns = self.namespace, self.annotations, {}

        self.namespace, self.annotations = ChainMap(class_ns, current_ns), ChainMap({}, current_ann)
        self.generic_visit(node)
        self.namespace, self.annotations = current_ns, current_ann

        class_def.methods.update(
            (name, value)
            for name, value in class_ns.items()
            if isinstance(value, Function)
        )

        return NoValue
```

pycon.it
~ BIARRITZ 2026

Appel de méthodes

- Pour appeler une méthode, il faut d'abord associer la fonction à l'instance
  - Cela permet le passage automatique de l'instance au paramètre `self`

In [95]:
```
def get_method(value, method_name):
    if not isinstance(value.type, Type):
        return NoValue
    function = value.type.methods.get(method_name, NoValue)
    if function is NoValue:
        return NoValue
    return function.bind(value)
```

pycon.fr
BIARRITZ 2026

Appel de méthodes

- La méthode est accédée comme un attribut de l'instance

In [96]:
```python
class TypingVisitor(...):
    [...]

    def visit_Attribute(self, node: ast.Attribute) -> PyObject:
        obj = self.visit(node.value)
        if isinstance(obj, PyObject) and isinstance(obj.type, Type):
            method = get_method(obj, node.attr)
            if method is not NoValue:
                return method
        self.add_error('Error', f'Unknown attribute {node.attr} on object of type {obj.type.name}', no
        return NoValue
```

pycon.fr
BIARRITZ 2026

Appel de méthodes

- Tout le reste fonctionne comme attendu car les méthodes sont des *callables*

In [97]:
```
%%writefile assets/methods.py
class A:
    def test(self: A) -> int:
        return 0

    def pouet(self: A, value: str) -> str:
        return value + '!'

a: A = A() # OK
b: int = a.test() # OK
c: str = a.test() # KO

d: str = a.pouet('test') # OK
e: int = a.pouet('test') # KO
a.pouet(5) # KO

a.tutu() # KO
```

```
Writing assets/methods.py
```

In [98]:
```
check_files('assets/methods.py')
```

```
Assignment error on assets/methods.py::10: Trying to assign value of type int to variable c annotated with str
Assignment error on assets/methods.py::13: Trying to assign value of type str to variable e annotated with int
Call Error on assets/methods.py::14: cannot call pouet with arg types int
Error on assets/methods.py::16: Unknown attribute tutu on object of type A
Call Error on assets/methods.py::16: cannot call  with arg types
```

pycon.fr
BIARRITZ 2026

Opérateurs en tant que méthodes

- Les opérateurs en eux-mêmes peuvent être vus comme des méthodes
  - C'est le cas dans le modèle objet de Python

In [99]:

```
[...]
builtins['int'].methods['__init__'] = Function(
    name='__init__',
    args=(builtins['int'], builtins['int'] | builtins['float'] | builtins['str']),
    returns=builtins['int'],
)
builtins['int'].methods['__add__'] = Function(
    name='__add__',
    args=(builtins['int'], builtins['int']),
    returns=builtins['int'],
)

builtins['float'].methods['__init__'] = Function(
    name='__init__',
    args=(builtins['float'], builtins['int'] | builtins['float'] | builtins['str']),
    returns=builtins['float'],
)
builtins['float'].methods['__add__'] = Function(
    name='__add__',
    args=(builtins['float'], builtins['float'] | builtins['int']),
    returns=builtins['float'],
)

builtins['str'].methods['__init__'] = Function(
    name='__init__',
    args=(builtins['str'], builtins['str'] | builtins['int'] | builtins['float']),
    returns=builtins['str'],
)
builtins['str'].methods['__add__'] = Function(
    name='__add__',
    args=(builtins['str'], builtins['str']),
    returns=builtins['str'],
)
```

pycon.fr
BIARRITZ 2026

Opérateurs plus complexes

- Attention, on entre maintenant dans les parties obscures de cette présentation
- On a besoin de quelques ajustements pour les opérateurs d'union (`__or__`) et d'appel (`__call__`)
    - Qui nécessitent d'exécuter du code pour déterminer leurs caractéristiques

Opérateurs plus complexes

- Attention, on entre maintenant dans les parties obscures de cette présentation
- On a besoin de quelques ajustements pour les opérateurs d'union (`__or__`) et d'appel (`__call__`)
    - Qui nécessitent d'exécuter du code pour déterminer leurs caractéristiques
- Une première fonction dont le type de retour (l'application) est défini à la volée

In [100]:
```python
import collections.abc

@dataclasses.dataclass(frozen=True)
class ApplicationFunction(Function):
    apply: collections.abc.Callable
```

pycon.fr
BIARRITZ 2026

Opérateurs plus complexes

- Et une seconde pour faire dépendre toutes les caractéristiques d'une classe

In [101]:

```
@dataclasses.dataclass(frozen=True)
class DynamicFunction(Callable):
    dynamic: collections.abc.Callable[[PyObject | None], Callable]

    @property
    def name(self):
        return self.dynamic(None).name

    @property
    def args(self) -> tuple[TypeForm, ...]:
        return self.dynamic(None).args

    @property
    def returns(self) -> TypeForm:
        return self.dynamic(None).returns

    def apply(self, *args: PyObject) -> PyObject:
        return self.dynamic(None).apply(*args)

    def bind(self, value: PyObject) -> DynamicMethod:
        return DynamicMethod(function=self, value=value)
```

pycon.fr
BIARRITZ 2026

Opérateurs plus complexes

- Avec son type de méthode associé

In [102]:
```python
class DynamicMethod(Method):
    function: DynamicFunction

    @property
    def args(self) -> tuple[TypeForm, ...]:
        return self.function.dynamic(self.value).args

    @property
    def returns(self) -> TypeForm:
        return self.function.dynamic(self.value).returns
```

pycon.fr
BIARRITZ 2026

Opérateurs plus complexes

- On redéfinit alors l'union entre types et l'instanciation (appel) de type

In [103]:

```
[...]
base_type.methods['__or__'] = DynamicFunction(
    lambda cls: ApplicationFunction(
        name='__or__',
        args=(base_type, base_type) if cls is None else (base_type,),
        returns=base_type,
        apply=lambda cls, rhs: UnionType(types=(cls, rhs)),
    ),
)
base_type.methods['__call__'] = DynamicFunction(
    lambda cls: ApplicationFunction(
        name='__call__',
        args=() if cls is None else cls.methods['__init__'].args[1:] if '__init__' in cls.methods else
        returns=cls,
        apply=lambda cls, *args: Value(type=cls),
    ),
)
```

pycon.fr
BIARRITZ 2026

Opérateurs plus complexes

- Idem pour l'appel de fonction

In [104]:
```
[...]
function_type.methods['__call__'] = DynamicFunction(
    lambda func: ApplicationFunction(
        name='__call__',
        args=() if func is None else func.args,
        returns=None if func is None else func.returns,
    ),
)
```

pycon.fr
BIARRITZ 2026

Opérateurs en tant que méthodes

- On associe les noms de méthodes aux opérateurs

In [105]:
```
operators = {
    ast.BitOr: '__or__',
    ast.Add: '__add__',
    ast.Call: '__call__',
}
```

pycon.fr
BIARRITZ 2026

Opérateurs en tant que méthodes

- On transforme la vérification des opérations pour faire appel aux méthodes associées

In [106]:

```python
def check_operation(visitor, operator, args, error_args):
    if not args or any(arg is NoValue for arg in args):
        return NoValue

    method_name = operators.get(operator)
    if method_name is None:
        visitor.add_error(*error_args)
        return NoValue

    value, args = args[0], args[1:]
    method = get_method(value, method_name)
    if method is NoValue:
        visitor.add_error(*error_args)
        return NoValue

    if method.match_args(*args):
        return method.apply(*args)

    visitor.add_error(*error_args)
    return NoValue
```

pycon.fr
BIARRITZ 2026

Opérateurs en tant que méthodes

- Et l'on branche tout ça dans le visiteur

In [107]:

```
class TypingVisitor(...):
    [...]

    def visit_BinOp(self, node):
        left = self.visit(node.left)
        right = self.visit(node.right)
        return check_operation(
            self,
            type(node.op),
            (left, right),
            error_args=(
                'Expression Error',
                f'Incompatible operation {type(node.op).__name__} for values of types {left.type.name}
                node,
            ),
        )

    def visit_Call(self, node):
        func = self.visit(node.func)
        args = tuple(self.visit(arg) for arg in node.args)
        return check_operation(
            self,
            ast.Call,
            (func, *args),
            error_args=(
                'Call Error',
                f'cannot call {getattr(func, 'name', '')} with arg types {", ".join(arg.type.name for a
                node,
            ),
        )
```

pycon ~ BIARRITZ 2026

Opérateurs en tant que méthodes

- Toujours compatible avec notre fichier d'exemple

In [ ]:
```python
# %load assets/operations.py
a: int = 3 + 5 # OK
a = 1 + 2 + 3 + 4 # OK
a = 3 + '5' # KO

b: str = 'a' + 'b' # OK
b = 3 + 2 # KO
```

In [109]:
```python
check_files('assets/operations.py')
```

```
Expression Error on assets/operations.py::3: Incompatible operation Add for values of types int and str
Assignment error on assets/operations.py::6: Trying to assign value of type int to variable b annotated with str
```

pycon.fr
BIARRITZ 2026



Conclusion

C’est un bon début

In [110]:
```
check_files(*Path('assets').glob('*.py'))
```

```
Assignment error on assets/functions.py::5: Trying to assign value of type int to variable b annotated with str
Call Error on assets/functions.py::11: cannot call addition with arg types int, str
Call Error on assets/functions.py::12: cannot call addition with arg types str, str
Assignment error on assets/methods.py::10: Trying to assign value of type int to variable c annotated with str
Assignment error on assets/methods.py::13: Trying to assign value of type str to variable e annotated with int
Call Error on assets/methods.py::14: cannot call pouet with arg types int
Error on assets/methods.py::16: Unknown attribute tutu on object of type A
Expression Error on assets/type_functions.py::4: Incompatible operation Add for values of types str and int
Assignment error on assets/classes.py::5: Trying to assign value of type A to variable b annotated with int
Assignment error on assets/classes.py::7: Trying to assign value of type int to variable c annotated with A
Assignment error on assets/classes.py::16: Trying to assign value of type str to variable g annotated with (A | in
t)
Assignment error on assets/classes.py::19: Trying to assign value of type A to variable i annotated with (D | int)
Assignment error on assets/classes.py::22: Trying to assign value of type str to variable l annotated with (D | in
t)
Expression Error on assets/operations.py::3: Incompatible operation Add for values of types int and str
Assignment error on assets/operations.py::6: Trying to assign value of type int to variable b annotated with str
Assignment error on assets/assignment.py::3: Trying to assign value of type str to variable a annotated with int
Assignment error on assets/assignment.py::7: Trying to assign value of type int to variable b annotated with str
Error on assets/assignment.py::9: b variable already defined
Assignment error on assets/assignment.py::13: Trying to assign value of type int to variable d annotated with str
Assignment error on assets/assignment.py::18: Trying to assign value of type str to variable g annotated with (int
| float)
```

pycon.fr
BIARRITZ 2026

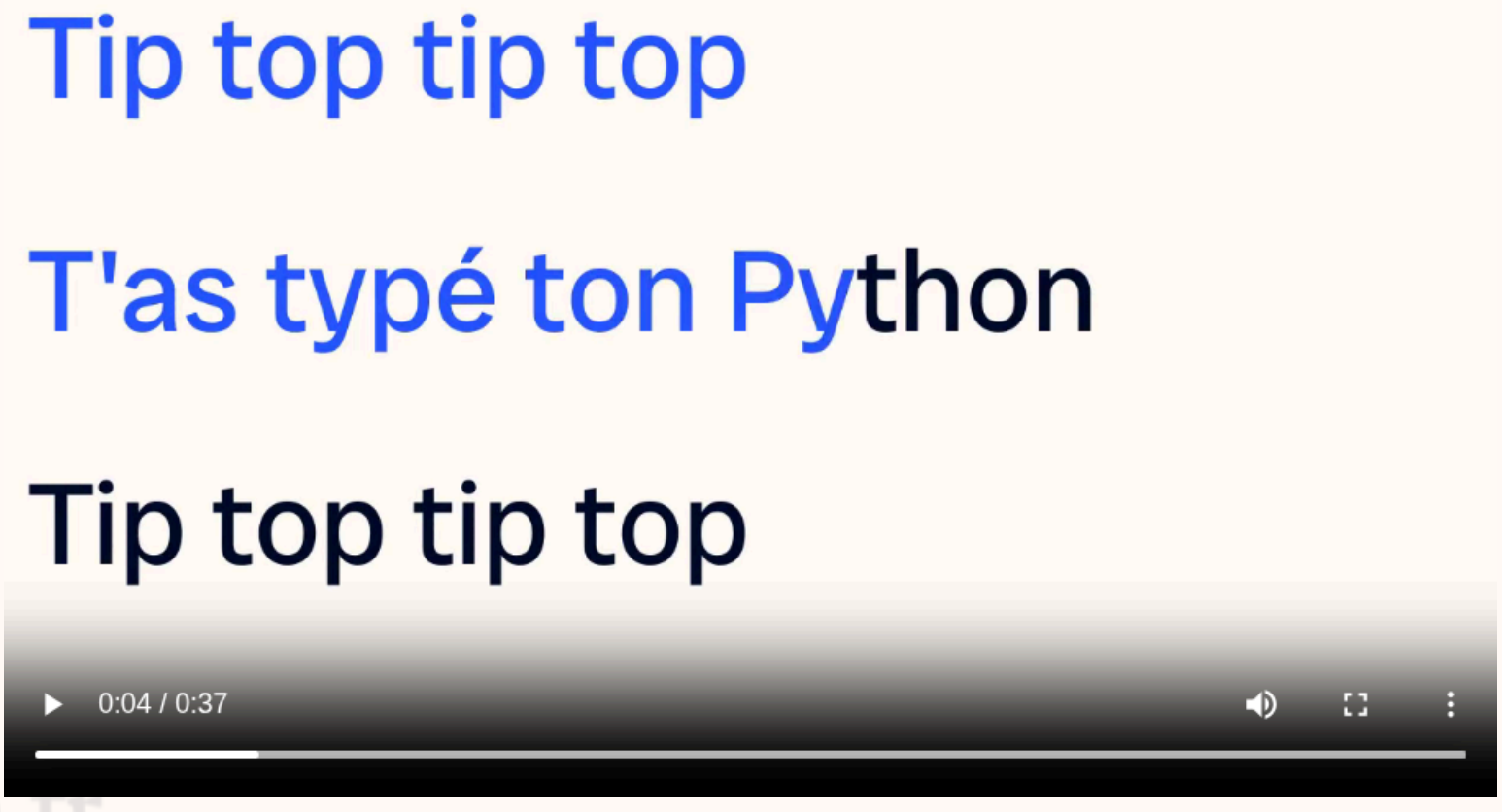
Mais très incomplet

- Fonctionnalités manquantes :
  - Gestion des attributs
  - Autres opérateurs, unaires ou binaires
  - Types composés et vérification récursive
  - Types génériques, protocoles, `Any`
  - Arguments variadiques (`*args`, `**kwargs`)
  - etc.

T’as typé ton Python

```
In [111]: karaoke()
```

Out[111]:



pycon.fr
BIARRITZ 2026

Conclusion

- http://github.com/entwanne/presentation_typing
  - Avec support et analyseur complet





pycon.fr
BIARRITZ 2026

Conclusion

- http://github.com/entwanne/presentation_typing
    - Avec support et analyseur complet





- Des questions ?

pycon.fr
BIARRITZ 2026